Prototipo Basado en IA para la Identificación de Transacciones Fraudulentas

Trabajo Final de Grado — Universidad Organización y Métodos (O&M)
Santiago de los Caballeros, República Dominicana

Descripción General

Este prototipo es una solución integral desarrollada en **Streamlit** que evalúa transacciones bancarias en tiempo real (de forma individual o por lotes) para identificar operaciones potencialmente fraudulentas. Combina cuatro modelos de aprendizaje automático (**Random Forest**, **XGBoost**, **Autoencoder** e **Isolation Forest**) con un motor híbrido de clasificación de riesgo en tres niveles (Bajo, Medio y Alto), e integra un módulo de explicabilidad mediante **SHAP** (Shapley Additive exPlanations).

Estructura del Proyecto y Guía de Navegación

La estructura de la carpeta del proyecto se organiza en 6 secciones principales para facilitar la navegación y evaluación técnica:

```
AIFraudesBancarios/
├── 1. codigo-fuente/           # Scripts Python, preprocesamiento, modelos,
SHAP, app Streamlit y requirements.txt
│   ├── preprocesamiento/       # Limpieza y generación de las 12
variables/features (src/preprocessing.py)
│   ├── modelos/                # Notebooks y scripts de entrenamiento:
Random Forest, XGBoost, Autoencoder, Isolation Forest
│   ├── shap/                   # Cálculo de explicabilidad en tiempo real
(SHAP)
│   ├── app/                    # Código de la aplicación Streamlit (app.py y
módulos auxiliares)
│   └── requirements.txt        # Librerías y versiones exactas usadas
│
├── 2. aplicacion/              # Aplicación ejecutable y pasos de puesta en
marcha
│   ├── README-EJECUCION.txt    # Pasos exactos para entorno virtual,
instalación y ejecución
│   └── run_app.bat             # Lanzador directo para la app en Windows
│
├── 3. data/                   # Datasets y exportación de resultados
│   ├── paysim_original/        # Dataset PaySim1 original en bruto
(raw/PS_20174392719_1491204439457_log.csv)
│   ├── paysim_procesado/       # Datasets limpios con 12 variables y arrays
de entrenamiento (.npy)
│   └── resultados/             # Transacciones clasificadas y reportes de
métricas en CSV (comparacion_modelos.csv, test_samples.csv)
│
```

```
├── 4. modelos/                 # Archivos serializados de los modelos
entrenados
│   ├── random_forest.pkl       # Modelo supervisado Random Forest
│   ├── xgboost.pkl             # Modelo supervisado XGBoost
│   ├── autoencoder.h5          # Red Neuronal Autoencoder (Keras/TensorFlow)
│   ├── autoencoder_config.pkl  # Configuración y umbral de reconstrucción
del Autoencoder
│   ├── isolation_forest.pkl    # Modelo no supervisado Isolation Forest
│   └── scaler.pkl              # StandardScaler para normalización de
características
│
├── 5. documentacion/           # Documentación técnica y manuales del TFG
│   ├── Manual_Usuario.pdf      # Manual de Usuario (Capítulo IV del TFG /
MANUAL_USUARIO.md)
│   └── README.md               # Este archivo general de explicación y
navegación
│
└── 6. evidencia/               # Evidencia de resultados y respaldos de
tesis
    └── graficos_shap/          # Gráficos SHAP, curvas ROC, matriz de
confusión e importancia de variables
```

Organización Detallada de los Componentes

1. Código fuente (`/codigo-fuente`)

Todo el código Python estructurado modularmente para el ciclo de vida del proyecto:

- **Preprocesamiento de datos (`/preprocesamiento`):** Código de transformación e ingeniería de características en `src/preprocessing.py` y `notebooks/02_Preprocesamiento.ipynb`. Genera las 12 variables predictivas clave (`error_balance_orig`, `error_balance_dest`, `ratio_monto_saldo`, `hora_dia`, `es_extraccion_alta`, etc.).
- **Entrenamiento de modelos (`/modelos`):** Notebooks de entrenamiento para los 4 algoritmos:
  - `notebooks/03_Modelo_IsolationForest.ipynb`
  - `notebooks/04_Modelo_RandomForest.ipynb` (con SMOTE)
  - `notebooks/05_Modelo_XGBoost.ipynb` (con SMOTE)
  - `notebooks/06_Modelo_Autoencoder.ipynb` (Deep Learning)
- **Cálculo de SHAP (`/shap`):** Integración de explicabilidad local para justificar la puntuación de riesgo de cada transacción en la interfaz.
- **Código de la app Streamlit (`/app`):** Ubicado en `app/app.py`. Contiene las 4 pestañas operativas (Análisis Individual & XAI, Análisis Masivo Batch, Dashboard Científico de Métricas y Consola de Analista Back-Office).
- **Dependencias (`requirements.txt`):** Especificación exacta de librerías y versiones instaladas (`streamlit`, `scikit-learn`, `xgboost`, `tensorflow`, `shap`, `plotly`, `pandas`, `numpy`, `imbalanced-learn`).

2. Aplicación ejecutable (`/aplicacion`)

La app Streamlit está configurada para ejecución directa sin requerir reentrenamiento:

- **Instrucciones completas (`README-EJECUCION.txt`):** Pasos detallados para configurar y desplegar la app.
- **Puesta en marcha paso a paso:**
  1. Crear un entorno virtual:
     ```
     python -m venv venv
     ```
  2. Activar el entorno virtual:
     - Windows: `venv\Scripts\activate`
     - macOS/Linux: `source venv/bin/activate`
  3. Instalar las dependencias exactas:
     ```
     pip install -r requirements.txt
     ```
  4. Iniciar la aplicación Streamlit:
     ```
     streamlit run app/app.py
     ```
- **Lanzador directo en Windows:** Se incluye el archivo `run_app.bat` para iniciar el entorno y la app con doble clic.

3. Bases de datos (`/datos`)

Archivos de datos utilizados en la investigación:

- **PaySim Original (`data/raw`):** Dataset PaySim1 (`PS_20174392719_1491204439457_log.csv`, 6.36 millones de registros, ~493 MB). *Nota: Si el archivo binario no se incluye físicamente por restricciones de peso, está disponible en Kaggle PaySim1.*
- **PaySim Procesado (`data/processed`):** Archivos `.npy` con el dataset limpio, normalizado y con ingeniería de variables para train/val/test (`X_train`, `X_test`, `y_train`, `y_test`).

4. Modelos entrenados (`/modelos`)

Modelos entrenados y serializados para uso inmediato en la app sin necesidad de reentrenamiento:

- `random_forest.pkl`: Random Forest (Supervisado, F1-Score: 99.84%).
- `xgboost.pkl`: XGBoost Classifier (Supervisado, F1-Score: 99.76%).
- `autoencoder.h5`: Red Neuronal Profunda Autoencoder (Semisupervisado en Keras).
- `autoencoder_config.pkl`: Umbral de corte del Error Cuadrático Medio (MSE) para el Autoencoder.
- `isolation_forest.pkl`: Isolation Forest (No supervisado, Detección de anomalías).
- `scaler.pkl`: Objeto `StandardScaler` ajustado para la normalización en tiempo real.

5. Documentación técnica (`/documentacion`)

Entregables de documentación requeridos en el Capítulo IV del Trabajo Final de Grado:

- **Manual de Usuario (`MANUAL_USUARIO.md`):** Guía práctica e ilustrada para la navegación en la app Streamlit, simulación de OTP y atención de alertas en el panel de analistas.
- **README.md:** Este documento guía principal de la estructura del proyecto.

6. Evidencia de resultados (`/evidencia`)

Evidencias empíricas y respaldos cuantitativos presentados en el Trabajo Final de Grado:

- **Gráficos SHAP y Evaluativos (`/graficos_shap`):** Importancia global y local de variables (SHAP), curvas ROC comparativas (`fig_curvas_roc.png`), matrices de confusión (`fig_matrices_confusion.png`) y análisis de correlación (`fig_correlacion.png`).

Resumen de Reglas de Clasificación de Riesgo Híbrida

El prototipo utiliza un motor híbrido de 3 niveles de riesgo con umbrales configurables (30% y 70%):

- **Riesgo Bajo (< 30%):** Transacción legítima aprobada automáticamente.
- **Riesgo Medio (30% - 70%):** Requiere verificación adaptativa (desafío OTP).
- **Riesgo Alto (> 70%):** Transacción bloqueada preventivamente y enviada a revisión en la consola de analista.